

Week 8 | 课时 2 | 索引资产化: Embedding、Index Release、增量更新与回滚边界

本课导航

不要只说“我建了向量索引”，要说清“这是哪一版索引资产”	1
这节课解决什么问题	1
参考学习时间	1
先看一张总图	1
1. Index Manifest 是索引资产的 contract	1
2. dry-run 不是可选项	2
3. 维度不一致必须 hard fail	2
4. Index Release 与回滚	3
本课最重要判断	3
自检清单	3

不要只说“我建了向量索引”，要说清“这是哪一版索引资产”

索引一旦进入生产，就必须像数据表一样有版本、报告、回滚边界和质量门禁。

这节课解决什么问题

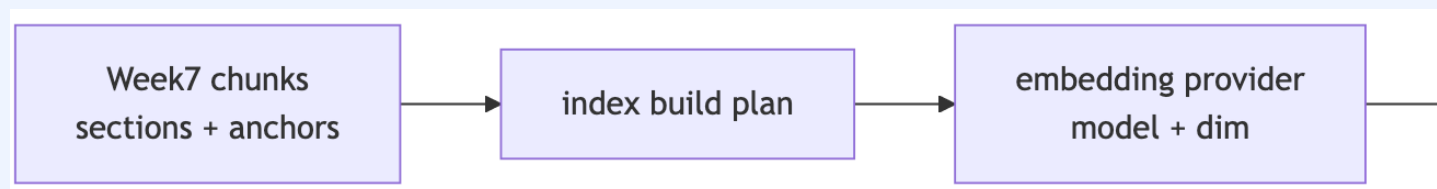
Week7 产出的是 evidence-ready chunks。Week8 首先要要把它们变成可管理的 index release:

- 使用哪一版数据?
- 使用哪一版 chunk strategy?
- 使用什么 embedding provider / model / dim?
- 哪些 chunk 被跳过?
- 维度不一致怎么办?
- 如何 dry-run 和回滚?

参考学习时间

45–55 分钟

先看一张总图



1. Index Manifest 是索引资产的 contract

最小字段:

字段	含义
<code>index_release_id</code>	这一版索引的稳定 ID
<code>data_release_id</code>	来源数据版本
<code>chunk_strategy_version</code>	chunk 策略版本
<code>embedding_model</code>	embedding 模型
<code>embedding_dim</code>	向量维度
<code>provider</code>	provider 或 fallback
<code>built_at</code>	构建时间
<code>total_chunks / embedded_chunks / skipped_chunks</code>	构建统计
<code>quality_gate</code>	是否允许进入服务

2. dry-run 不是可选项

Index build 的 dry-run 至少要回答：

- 本次会处理多少 chunks?
- 是否有 Week7 placeholder?
- provider key 是否可用?
- embedding dim 是否匹配数据库列?
- 会写入哪个 `index_release_id`?

```
python -m pipelines.indexing.embedder \
  --index-release-id index-week08-dev \
  --batch-size 32 \
  --dry-run
```

3. 维度不一致必须 hard fail

如果数据库列是 `vector(1536)`，provider 返回 384 或 1024 维，不能默默写入。

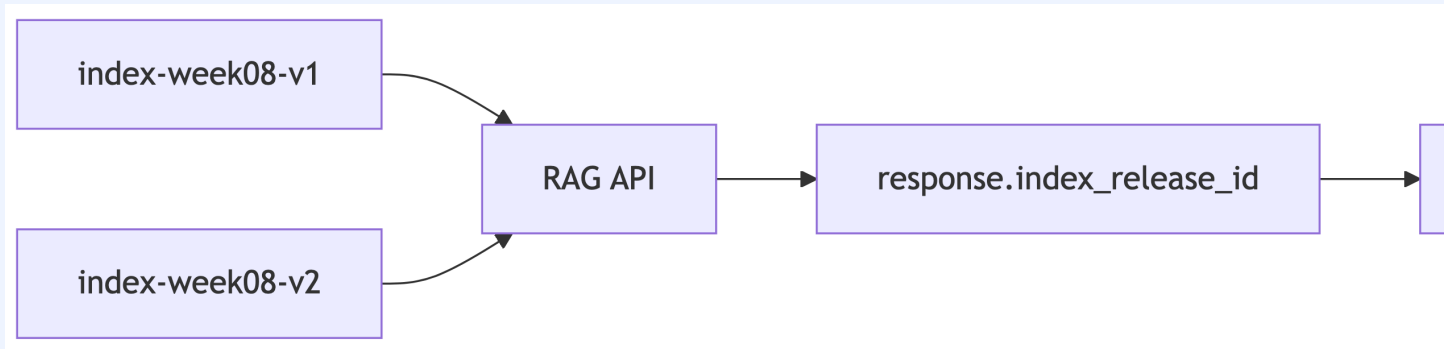
正确做法：

- 拒绝写入
- 报告 `dimension mismatch`
- 记录 provider、model、expected dim、actual dim
- 让学生知道这是 schema / provider contract 失配

⚠ 不要做的事

不要在代码里偷偷截断或 padding embedding。那会让检索质量问题变成不可解释的系统隐患。

4. Index Release 与回滚



当一次回答出错时，`index_release_id` 能帮助你判断：

- 是数据变了？
- 是 chunk strategy 变了？
- 是 embedding model 变了？
- 是 retrieval/rerank/prompt 变了？

本课最重要判断

索引不是缓存文件，而是 RAG 系统的可版本化资产。没有 index manifest，就没有可复现的 RAG 回答。

自检清单

- 我能解释为什么 index build 要有 dry-run。
- 我能说清 embedding dimension mismatch 为什么不能自动吞掉。
- 我能读懂 `index_manifest.schema.json`。
- 我知道 `index_release_id` 应该进入 RAG response 和 audit log。